

JPA 2.1 StoredProcedureQuery

JPA 2.1 supports calling database stored procedures using the `StoredProcedureQuery`, and `@NamedStoredProcedureQuery` annotation or `<named-stored-procedure-query>` XML element. JPA supports both named stored procedures calls defined in meta-data and created through `EntityManager.createNamedStoredProcedureQuery()`, and dynamic stored procedure calls created through `EntityManager.createStoredProcedureQuery()`.

`StoredProcedureQuery` is a JPA Query that provides additional API for setting the stored procedure parameters, and for accessing output parameters and multiple result sets. A `StoredProcedureQuery` can return entity objects, or data, similar to native SQL queries. A `ResultSetMapping` can be used to map the returned fields to the entity columns.

Some databases such as MySQL, SQL Server and Sybase support stored procedure that return result sets. Some databases such as Oracle do not, but do support output parameters that return cursors. If the stored procedure returns a result set, or a cursor output parameter, `getResultList()` or `getSingleResult()` can be used. If the stored procedure has multiple result sets, or multiple cursor output parameters, then calling `getResultList()` again will return the next result set or cursor output parameter. If the stored procedure does not return anything, `executeUpdate()` can be used. `StoredProcedureQuery` also defines an `execute()` API, that executes the procedure and returns a `boolean` indicating if a result set was returned. After execution, any of the stored procedures output parameters can be accessed using `getOutputParameterValue()`.

The stored procedure parameters are defined through the `@StoredProcedureParameter` annotation or `<stored-procedure-parameter>` XML element, or for dynamic stored procedure calls through the `StoredProcedureQuery.registerStoredProcedureParameter()` API. Parameters can be named or by index, define the corresponding Java type for the parameter, and define the parameter directional mode through `ParameterMode`. The mode can be one of `IN`, `INOUT`, `OUT`, `REF_CURSOR`. A type of `void.class` can be used for ref cursors.

Example named stored procedure annotation

```
// This stored procedure returns a result set and has one input parameter.
@NamedStoredProcedureQuery(
    name = "ReadAddressById",
    resultClasses = Address.class,
    procedureName = "READ_ADDRESS",
    parameters = {
        @StoredProcedureParameter(mode=IN, name="P_ADDRESS_ID",
type=Long.class)
    }
)
@Entity
public class Address {
    ...
}
```

Example calling a named stored procedure

```
StoredProcedureQuery query =
em.createNamedStoredProcedureQuery("ReadAddressById");
query.setParameter("P_ADDRESS_ID", 12345);
List<Address> result = query.getResultList();
```

Example named cursored stored procedure annotation

```
// This stored procedure returns a cursor output parameter, and has one input
parameter.
@NamedStoredProcedureQuery(
    name = "ReadAddressById",
    resultClasses = Address.class,
    procedureName = "READ_ADDRESS",
    parameters = {
        @StoredProcedureParameter(mode=IN, name="P_ADDRESS_ID",
type=Long.class)
        @StoredProcedureParameter(mode=REF_CURSOR, name="CUR_ADDRESS",
type=void.class)
    }
)
@Entity
public class Address {
    ...
}
```

Example calling a named stored procedure with a cursor output parameter

```
StoredProcedureQuery query =
em.createNamedStoredProcedureQuery("ReadAddressById");
query.setParameter("P_ADDRESS_ID", 12345);
query.execute();
List<Address> result =
(List<Address>) query.getOutputParameterValue("CUR_ADDRESS");
```

Example calling a dynamic stored procedure

```
StoredProcedureQuery query =
em.createNamedStoredProcedureQuery("VALIDATE_ADDRESS");
query.registerStoredProcedureParameter("P_COUNTRY", String.class,
ParameterMode.IN);
query.registerStoredProcedureParameter("P_PROVINCE", String.class,
ParameterMode.IN);
query.registerStoredProcedureParameter("P_CITY", String.class,
ParameterMode.IN);
query.registerStoredProcedureParameter("P_STREET", String.class,
ParameterMode.IN);
query.registerStoredProcedureParameter("P_POSTAL_CD", String.class,
ParameterMode.IN);
query.registerStoredProcedureParameter("OUT_VALID", Boolean.class,
ParameterMode.OUT);
query.registerStoredProcedureParameter("OUT_ID", Long.class,
ParameterMode.OUT);
```

```

query.setParameter("P_COUNTRY", "Canada");
query.setParameter("P_PROVINCE", "ON");
query.setParameter("P_CITY", "Ottawa");
query.setParameter("P_STREET", "99 Bank");
query.setParameter("P_POSTAL_CD", "K2H8L2");

query.execute();
Boolean isValid = (Boolean)query.getOutputParameterValue("OUT_VALID");
Long id = (Long)query.getOutputParameterValue("OUT_ID");

```

Stored Procedures in JPA 2.0

JPA 2.0 does not have any direct support for stored procedures. Some types of stored procedures can be executed in JPA through using native queries. Native queries in JPA allow any SQL that returns nothing, or returns a database result set to be executed. The syntax to execute a stored procedure depends on the database. JPA native SQL queries do not support stored procedures that use `OUTPUT` or `INOUT` parameters. Some databases such as DB2, Sybase and SQL Server allow for stored procedures to return result sets. Oracle does not allow results sets to be returned, only `OUTPUT` parameters, but does define a `CURSOR` type that can be returned as an `OUTPUT` parameter. Oracle also supports stored functions, that can return a single value. A stored function can normally be executed using a native SQL query by selecting the function value from the Oracle `DUAL` table.

Some JPA providers have extended support for stored procedures, some also support overriding any CRUD operation for an `Entity` with a stored procedure or custom SQL. Some JPA providers have support for `CURSOR OUTPUT` parameters.

[TopLink](#) / [EclipseLink](#) : Support stored procedures and stored functions using the `@NamedStoredProcedureQuery`, `@NamedStoredFunctionQuery` annotations or XML, or the `StoredProcedureCall`, `StoredFunctionCall` classes. Overriding any CRUD operation for a class or relationship are also supported using a `DescriptorCustomizer` and the `DescriptorQueryManager` class. `IN`, `OUT`, `INOUT`, and `CURSOR OUTPUT` parameters are supported.

Example executing a stored procedure on Oracle

```

EntityManager em = getEntityManager();
Query query = em.createNativeQuery("BEGIN VALIDATE_EMP(P_EMP_ID=>?); END;");
query.setParameter(1, empId);
query.executeUpdate();

```

PL/SQL Stored Procedures

In Oracle stored procedures are typically written in Oracle's PL/SQL language. PL/SQL in Oracle supports some additional data-types, that Oracle does not support through SQL or JDBC. These include types such as `BOOLEAN`, `TABLE` and `RECORD`. Accessing these types or procedures is difficult from Java, as these types are not supported by JDBC. One workaround is to wrap the

PL/SQL stored procedures with normal stored procedures that transform the PL/SQL types to standard SQL/JDBC types, such as `INT`, `VARRAY (Array)`, and `OBJECT TYPE (Struct)`. Some JPA providers have extended support for calling PL/SQL stored procedures.

[TopLink](#) / [EclipseLink](#) : Support PL/SQL stored procedures and functions using the `@NamedPLSQLStoredProcedureQuery`, `@NamedPLSQLStoredFunctionQuery` annotations or XML, or the `PLSQLStoredProcedureCall`, `PLSQLStoredFunctionCall` classes.